ScholarPulse

Final Project Report | Milestones 1–6

Project: ScholarPulse: AI-Enabled Academic Knowledge Hub

Course: Course Project 2026 — Theme 4: Knowledge Hub

Team: Harshit Pant, Snehadip Ghoshal, Gangadevi Venkata Siva Krishna, Ritik Kumar

Executive summary

ScholarPulse is a course-oriented academic knowledge hub for discovering, organizing, rating, commenting on and moderating academic resources. Its design includes a grounded AI Study Assistant based on retrieval-augmented generation (RAG), with resource, collection and course scopes.

The repository package contains a FastAPI backend, SQLAlchemy data layer, MySQL schema and Alembic migrations, seed scripts, OpenAPI documentation, Docker Compose configuration, frontend UI, backend integration tests, CI workflow and Playwright smoke tests.

This report distinguishes implemented repository artifacts from work that still requires external credentials, human review or live-environment verification. It does not claim that remote CI, manual accessibility review, Figma submission, reviewed pull requests or live Gemini calls have passed unless evidence is attached.

1. Problem and objectives

Academic resources are often scattered across notes, paper summaries and shared drives. ScholarPulse provides a shared, course-organized catalogue with discovery, organization, community feedback and moderation. Its AI feature is intended to answer questions from selected resources and show source citations, while refusing when the available material does not support an answer.

2. Users and roles

- Visitor: browse, search and read public resources.
- Student: upload permitted resources, rate, comment, ask questions, create collections and report content.
- Teaching assistant: publish official resources and reading lists.
- Moderator: review reports and hide, restore, remove or dismiss reported content.

3. Architecture and technology

- Frontend: HTML, CSS and JavaScript.
- Backend: Python FastAPI.
- Validation: Pydantic.
- Persistence: MySQL 8.0 and SQLAlchemy.
- Migrations: Alembic.

• Authentication: Argon2id password hashing, server-side session cookie and CSRF protection.

• AI design: chunking, retrieval, context construction, answer generation and citation validation.

• Testing: Pytest, pytest-cov and Playwright.

• Runtime: Docker Compose and Uvicorn.

• API contract: OpenAPI 3.0.3 and Swagger UI.

4. Data model

The documented application schema has 12 principal tables: users, categories, resources, resource_chunks, tags, resource_tags, ratings, comments, collections, collection_items, reports and ai_queries. The model includes one-to-many relationships, many-to-many association tables, uniqueness constraints and indexes for common catalogue and retrieval operations.

5. Milestone summary

MS-1 — Problem statement and design

The project definition, user roles, user stories, dataset plan, architecture, risks and team responsibilities are documented. The committed seed subset and team-authored notes are included in the repository package.

MS-2 — Prototype and data design

The ER diagram and database/index documentation are included. The Figma prototype export/share evidence and signed accessibility verification must be attached separately. The accessibility checklist is intentionally marked pending until keyboard, focus, contrast, labels and reflow checks are performed on the live UI.

MS-3 — API and backend foundation

The package includes OpenAPI YAML, FastAPI routes, SQLAlchemy models, Alembic migration revisions, seed scripts, Docker Compose and backend tests. The root OpenAPI file is synchronized with docs/openapi.yaml.

MS-4 — Testing and frontend integration

The package includes a CI workflow with a 60% coverage gate, backend test command, Playwright configuration and four browser smoke tests. These are configured artifacts. A passing remote GitHub Actions run must be captured after pushing the package. A source ZIP cannot prove reviewed pull requests.

MS-5 — Feature implementation and AI

The backend source includes authentication, role checks, profile operations, resource upload and extraction, ratings, comments, collections, reports, moderation, and AI question-answering endpoints with resource/collection/course scope support in the documented API. The current RAG service in this package uses deterministic local retrieval/extractive synthesis and keyword fallback. Production Gemini generation and Gemini embedding calls are not verified as active in this package. They require server-side API credentials, integration testing, quota/timeout handling and evaluation evidence. Do not present the local baseline as a completed live Gemini integration.

MS-6 — Final report, presentation, demo and viva

This report, presentation deck, demo runbook and viva preparation materials are included. The final live demonstration, team sign-off and external evidence collection remain actions for the team.

6. Security and privacy

The design uses salted Argon2id password hashes, HTTP-only session cookies, CSRF protection for state-changing operations, role-based authorization, server-side validation and upload restrictions. Public document text may be sent to a configured external AI service only when the team enables that integration and accepts the provider's applicable terms. Secrets must remain in environment variables or GitHub Secrets, never in source control or frontend code.

7. Testing and quality strategy

• Backend integration tests exercise API behavior, authentication, authorization, uploads, retrieval/fallback and moderation.

• Coverage is enforced at a minimum of 60% in the updated CI workflow.

• Playwright smoke tests cover home-page loading, health, paginated catalogue API and search API.

• RAG evaluation uses a ground-truth question set and reports retrieval recall, refusal, citation, multi-document coverage and prompt-injection behavior.

• Manual accessibility checks and exploratory feature tests must be recorded separately.

8. Known limitations and release gates

1. Live Gemini embedding and generation require valid server-side credentials and must be verified with real API calls.

2. Figma prototype export/share access and a short walkthrough recording must be attached.

3. Accessibility checklist remains pending until manual tests are completed and evidence is saved.

4. GitHub Actions must be run remotely and its successful run URL recorded.

5. Reviewed/approved pull requests must be linked.

6. Any feature not exercised in the live demo should be described as implemented in source, partially integrated or pending—not as production verified.

9. Demo sequence

1. Start Docker Compose and show healthy database/backend services.

2. Show Alembic current revision and seed counts.

3. Open the home page and Explore catalogue.

4. Search for a known term and demonstrate pagination/filtering.

5. Open a resource detail page.

6. Sign in using a local demo account.

7. Demonstrate a permitted rating/comment or collection operation.

8. Demonstrate upload validation using a small text/PDF file.

9. Show role-restricted moderation with the moderator account.

10. Ask an AI question in resource, collection or course scope; show citations or fallback notice.

11. Open Swagger UI and explain one endpoint's request/response contract.

12. Show test and CI reports, clearly distinguishing local from remote results.

10. Conclusion

ScholarPulse establishes a course-based resource hub with a documented relational model, API contract, backend services, frontend interface and verification workflow. The project should be considered submission-ready only after the external evidence and live verification gates above are completed. The team should report the current deterministic RAG baseline honestly and treat live Gemini integration as a separate verified release gate.